

CSEC5616/CSEC3616 — S2 2026
Assignment - 2

This is an **individual** assignment.

This assignment worths 10% of the final marks of the course. It covers Weeks 4-6 (inclusive).

Submit your final report as a PDF and codes as a zip file in Canvas. In Canvas, under Assignment 2, you will find two links to submit your report and code separately.

You should explain any details of how to run your code in the report.

Please read the below instructions carefully.

***** IMPORTANT *****

1) **Note the answer release date:** Please note the answer release date mentioned below. Any submission after that will get zero marks, instead of a late penalty.

2) **Typeset reports only:** We accept only typeset answers. Any hand written answers will get zero marks. This is because we can't do plagiarism checks for hand written answers.

3) **DO NOT repeat questions in the report:** Simply include the question number and your answer only. If you include question text in your answer sheet, your TurnItIn score will be high and there will be additional checks. This will cause a delay in releasing your marks. **We will also impose a penalty of 10% of the total marks.**

3) **Cite your sources:** If you are referring to any internet sources include them as citations. We do not expect any specific citation style. You are free to select a style you think as appropriate. As mentioned in class announcements you are free to use GenAI. If you do so, make declaration in the report. If you are using material from the lecture slides or lecture notes, you do not need to cite them explicitly.

***** SUBMISSION *****

Final Report & Code: Due by Week 8, Sunday the 27th of September, 2026 11:59PM

Informal Extensions: For this assignment, you can submit up to 5 days late (i.e., until **Friday 02/10/2026 23:59**) without any late penalty. You do not have to lodge any request through the Special Considerations portal or contact the unit staff for this. Everyone is automatically entitled to this. Please note that this extension is not updated in Canvas, so it is normal if you see a late submission flag in Canvas. After this extension period, late penalties will apply as per the university policy. Please refer to the Administrivia slide in Week 1.

Answer release: The answers to this assignment will be automatically released on **05 October 2026 at 00:00**. Any submissions after that will get **zero marks**. If you have a legitimate reason that requires an extension beyond that you will need to go through the university

[special considerations process](#). If approved, what you will be granted is a mark adjustment not an extension.

1 Breaking the Vigenère Cipher (25 marks)

In the given zip file for Assignment 2 (Question1.zip), you will find a cipher text encrypted by the Vigenère cipher. Your task is to break it and find the original plain text. Please read the following instructions carefully.

- You need to write your own code for this. However, there will be some guesswork and analysis involved. So we do not expect your code to give the decrypted result straight away. For example, if you are checking various substitutions, it is acceptable that you run your code multiple times to see which substitution results in a readable text.
- The **spaces** and punctuations such as . and ' are not encrypted. You can store their positions into an array, remove them, do the decryption using your method, and insert them back to the correct positions in the final plaintext.
- You will need ideas from **Kasiski Test** to break this. However there can be other possible solutions. You are not allowed to use any online decryption tool to assist you.
- The key length is less than 20 characters and contain only capital English characters. It is a random string, not a meaningful word. The final plain text is readable and meaningful.
- In the report include a description of your approach, the key length, the key, and the final plaintext you obtained. Also, explain how your code works and how to run it so that the tutors can run your code.

2 AES Calculation (25 marks)

In this question, you will perform some AES calculations manually. You are given the following information in hexadecimal notation. Here, "manually" means performing the calculations yourself step by step - without using any programs or scripts. Note that your submission must be typed; we do not accept handwritten work, even if the calculations are done by hand.

- Plaintext 0C0B0A090807060504030201000F0E0D
- First round key 05050505050505050505050505050505.

Specifically, you will be **manually** calculating the initial steps in the AES calculation as illustrated in Figure 1.

Answer the following questions. For each step you need to explain what you are doing, your calculation steps, and the answer.

Note: We do not expect fine-granular step for each individual calculation. For example, it is ok to show detailed steps of one of the mix-column calculations and show the answers for other calculations.

- i Show the original contents of State, displayed as a 4×4 matrix. Enter the bytes column-wise, i.e., fill the first column from top to bottom, then the second column, and continue in order. **(1 marks)**
- ii Show the value of State after initial AddRoundKey. **(3 marks)**

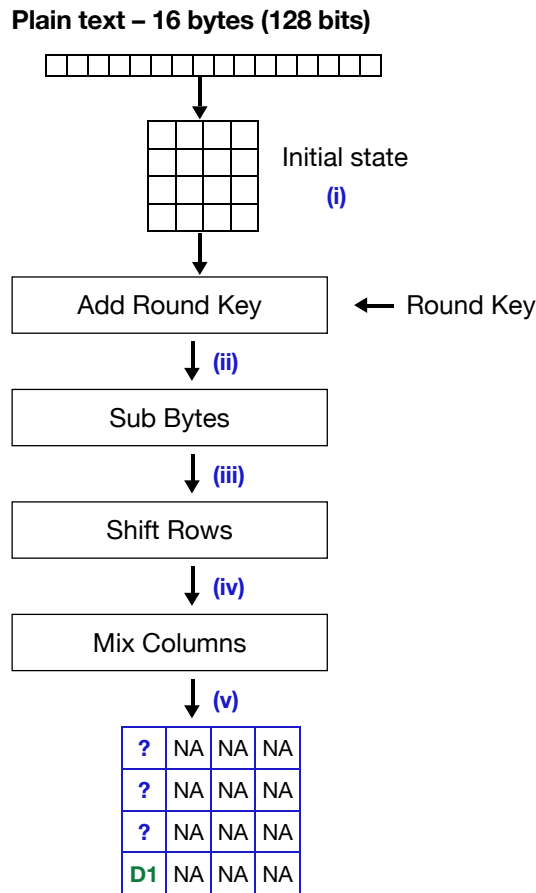


Figure 1: AES manual calculation

iii Show the value of State after SubBytes. **(3 marks)**

iv Show the value of State after ShiftRows. **(3 marks)**

v Calculate and show the missing values of the first column of the State after MixColumns (see the last matrix in Figure 1). (You can use the remaining value of the column to check whether you did the correct calculations in the previous steps. The rest of the values of the state marked as NA are not applicable to the question.) **(15 marks)**

Note: Addition and multiplication for AES are done on $GF(2^8)$ with the irreducible polynomial $m(x) = x^8 + x^4 + x^3 + x + 1$.

Referring to following material will be helpful to complete this task

- [AES Rijndael Cipher explained as a Flash animation](#)
- AES wiki page [Advanced Encryption Standard](#)
- Cryptography and Network Security (Seventh Edition - William Stallings) - Chapter 6

3 RSA Calculation (25 marks)

Suppose Bob is generating a public and private key pair using RSA algorithm. Bob chooses the prime numbers $p = 4,201$ and $q = 5,701$. He computed $n = pq = 23,949,901$ and $\phi(n) = (p-1)(q-1) = 23,940,000$. He then chooses the encryption exponent $e = 65,537$.

- i Find the private key. i.e., d where $ed \equiv 1 \pmod{\phi(n)}$ using the Extended Euclidean algorithm **(14 marks - 11 marks for the missing values in the table and 3 marks for d)**.

First follow the explanation below.

Recall the Euclidean Algorithm for finding the GCD of a and b . It goes like this

1) Calculate $r_1 = a \pmod{b}$ which satisfies $a = q_1b + r_1$

2) Calculate $r_2 = b \pmod{r_1}$ which satisfies $b = q_2r_1 + r_2$

3) Calculate $r_3 = r_1 \pmod{r_2}$ which satisfies $r_1 = q_3r_2 + r_3$

...

i) Calculate $r_i = r_{i-2} \pmod{r_{i-1}}$ which satisfies $r_{i-2} = q_i r_{i-1} + r_i$

...

n) Calculate $r_{n+1} = r_{n-1} \pmod{r_n} = 0$ which satisfies $r_{n-1} = q_{n+1}r_n + 0$

When r_{n+1} becomes zero $\gcd(a, b) = r_n$

Next we assume that each step we can find x_i and y_i that satisfy $r_i = ax_i + by_i$. We end up with the following sequence.

$$a = q_1b + r_1 \text{ and } r_1 = ax_1 + by_1$$

$$b = q_2r_1 + r_2 \text{ and } r_2 = ax_2 + by_2$$

$$r_1 = q_3r_2 + r_3 \text{ and } r_3 = ax_3 + by_3$$

$$r_{n-2} = q_nr_{n-1} + r_n \text{ and } r_n = ax_n + by_n$$

$$r_{n-1} = q_{n+1}r_n + 0$$

We can rearrange the terms in above equations to obtain

$$r_i = r_{i-2} - r_{i-1}q_i \tag{1}$$

But we also know that $r_{i-2} = ax_{i-2} + by_{i-2}$ and $r_{i-1} = ax_{i-1} + by_{i-1}$

By substituting in Equation (1) we get.

$$r_i = (ax_{i-2} + by_{i-2}) - (ax_{i-1} + by_{i-1})q_i$$

$$r_i = a(x_{i-2} - q_ix_{i-1}) + b(y_{i-2} - q_iy_{i-1})$$

Since we assumed $r_i = ax_i + by_i$ by co-efficient matching we can get

$$x_i = x_{i-2} - q_ix_{i-1} \text{ and } y_i = y_{i-2} - q_iy_{i-1}$$

Based on this information your task is to fill the blanks in Table 1 and find the value of d .

When marking, we do not expect fully worked-out details for every step in the table. You must explain the basic idea you used to fill the table and provide full details for the second step (i.e., $i = 2$ in the table). We also expect some explanation of how you selected the final d value.

Table 1: Computing the decryption exponent d .

i	q_i	r_i	x_i	y_i
-1	NA	23,940,000	1	0
0	NA	65,537	0	1
1	365	18,995	1	-365
2	-	-	-	-
3	-	-	-	-
4	-	-	-	-
5	-	-	-	-
6	-	-	-	-
7	-	-	-	-
8	-	-	-	-
9	-	-	-	-
10	-	-	-	-
11	-	-	-	-
12	-	-	-	-
13	10	0	NA	NA

-
- ii Suppose Bob shared his public key e with Alice. If Alice wants to send the message 3 to Bob, find the encrypted message. Show your steps of exponentiation by squaring (You don't have to show all the steps but you can show first 2-3 steps of the exponential squaring and show the final answer - you can use an online calculator to find the final answer e.g. <https://www.dcode.fr/modular-exponentiation>). (6 marks).
- iii Verify that Bob can decrypt what Alice sends. Describe the required computation and use an online calculator to obtain the final answer. (5 marks).

4 Message Authentication Codes (25 marks)

a) Message Authentication Codes vs. Hashes

- i Explain what Authenticated Encryption (AE) is and why it is required. (2 marks)
- ii Explain the difference between Authenticated Encryption (AE) and Authenticated Encryption with Associated Data (AEAD) (2 marks)

b) Protocol Analysis

Consider the following two protocols that are used by Alice to send a message to Bob.

Protocol X: $y = E_{k_1}(x || H(k_2 || x))$

Here x is the plaintext message, k_1 and k_2 are shared keys between Alice and Bob, and H is a cryptographically secure hashing function such as SHA256. E stands for a AES encryption. Once y is computed Alice sends y to B. (Here, $||$ denotes concatenation.)

Protocol Y: $x, y = E_{PK}(H(x))$

Here, PK is the public key of Bob and it is assumed Bob has access to the corresponding private key SK . H is again a cryptographic secure hashing function such as SHA256. E stands for RSA encryption. Once y is computed Alice sends x and y to Bob.

- i Explain step by step what Bob will do after the reception of the message y in each of the protocols. (4 marks)
- ii Explain whether confidentiality and integrity is achieved in each of the two protocols. (4 marks)

c) Flawed Hash

This problem explores a letter-based hash function called the **Miniature Letter Hash (MLH)**. Like standard cryptographic hash functions, MLH takes a message of arbitrary length and produces a fixed-size digest, in this case, a sequence of four letters.

1) Preprocessing: Extract only the letters from the input message, discarding everything else, and convert them to uppercase. Then divide what remains into blocks of 16 letters. If the final block has fewer than 16 letters, pad it with nulls (i.e., 0 - zero). A four-number accumulator is initialized to (0, 0, 0, 0) before processing begins.

Accumulator: A four-number accumulator is initialized to (0, 0, 0, 0) before processing begins. After each round, the accumulator is updated using: $\text{new_accumulator}[i] = (\text{column_sum}[i] + \text{old_accumulator}[i]) \bmod 26$, for $i = 1, 2, 3, 4$. The updated accumulator is then carried into the next round. After both rounds of the final block are complete, the accumulator values are converted back to letters ($0 = A, 1 = B, \dots, 25 = Z$) to produce the hash.

2) Processing each block: Every block is passed through a compression function consisting of two sequential rounds. The accumulator entering a block's compression is the one output by the previous block (or (0, 0, 0, 0) for the first block). After both rounds complete, the updated accumulator is carried forward as the input to the next block's compression. This continues until all blocks are processed, at which point the final accumulator values are converted to letters to form the hash.

2a) Round 1: Arrange the 16-letter block into a 4×4 grid, filling row by row left to right. Assign each letter a numeric value ($A = 0, B = 1, \dots, Z = 25$). Sum each column modulo 26 to produce four intermediate values. Add each intermediate value to the corresponding position of the incoming accumulator, modulo 26. The resulting four values form the updated accumulator, which is passed into Round 2. See Figure 2.

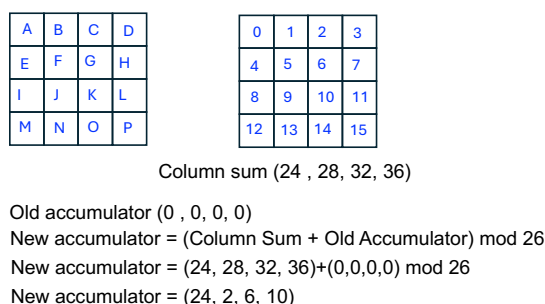


Figure 2: MLH Round 1 - Example

2b) Round 2: Using the same grid from Round 1, apply the following row transformations: shift row 1 left by one position, shift row 2 left by two positions, shift row 3 left by three positions, and reverse row 4. Sum each column of this transformed grid modulo 26 to produce four intermediate values. Add each to the corresponding position of the Round 1 accumulator, modulo 26. The result is the accumulator passed to the next block's compression, or the final hash accumulator if this was the last block. See Figure 3.

- i Draw a figure comparable to the figure in Week 6 - Slide 26 to depict the overall MLH logic and the compression function logic. **(2 marks)**
- ii Calculate the hash function for the 32-letter message "Hello Dave everything is running well." **(3 marks)**
- iii To demonstrate the weakness of MLH, find a message of length 22-letter that produces the same

B	C	D	A
G	H	E	F
L	I	J	K
P	O	N	M

1	2	3	0
6	7	4	5
11	8	9	10
15	14	13	12

Column Sum (33, 31, 29, 27)

Old Accumulator (24, 2, 6, 10)

New Accumulator = (Column sum + Old Accumulator) mod 26

New Accumulator = (24, 2, 6, 10) + (33, 31, 29, 27) mod 26

New Accumulator = (5, 7, 9, 11)

If the original message is only ABCDEFGHIJKLMNOP we can get the hash by converting the new accumulator back to characters.

That is (5, 7, 9, 11) = FHJL

If there are other blocks in the original message, the new accumulator value goes to next block.

Figure 3: MLH Round 2 - Example

hash. Explain your thought process of deriving this text. (There can be more than one answer)
(8 marks)